Java Blueprints

Промежуточный итог: соответствие проекта кейсу Axenix «Визуальная среда программирования»

Дата анализа: 24.10.2026 • Репозиторий: ветка main, 18 коммитов • Контрольная точка: 08:00 25.10 — ссылки на репозитории HR (@ekaterina_axenix)

Короткий вывод. Бэкенд закрывает почти весь контракт кейса: разбор Java в граф, правка графа с обратной генерацией кода, диагностика Big-O, декомпозиция монолита и Quality Gate с потоковым отчётом. Главный риск — клиент. Интерфейс визуально соответствует концепту, но основной путь **Java ↔ граф** в нём не работает: граф, проблемы и метрики берутся из заготовленных данных (mockData.ts), а из 10 эндпоинтов бэкенда интерфейс вызывает только 4. Часть результатов на экране (12 из 12 тестов, ускорение 5.2x) прошита в коде, а не получена от сервера. До сдачи нужно связать клиент с бэкендом и убрать имитацию результатов.

Бэкенд	Клиент	Тесты	Сдача
Основная логика реализована; 63 тестовых метода в 8 классах; Java 21, Spring Boot 3.3	Оболочка готова; граф, проблемы, метрики — моки; 4 из 10 эндпоинтов подключены	Только сервер (CI на GitHub Actions). Тестов клиента нет. Сохранённого отчёта прогона нет	Код и запуск есть; описание решения устарело и завышает готовность

1. Что требует кейс

Команда за 48 часов создаёт веб-инструмент, который понимает Java-монолит (≈1000 строк) как код и как граф, помогает улучшить его и доказывает сохранение поведения.

- **Ключевой контракт:** после любых преобразований автотесты подтверждают, что поведение программы сохранилось.
- **Путь пользователя (6 шагов, финиш обязателен):** загрузить .java → разобрать → построить граф → исследовать и править → собрать Java → запустить и сравнить. Изменения синхронизируются в обе стороны.
- **Диагностика:** строка кода и узел графа, объяснение дефекта, оценка сложности (Big O), вариант исправления с ожидаемым эффектом. Диагноз должен быть проверяемым.
- **Оптимизация и декомпозиция:** при необходимости выделить Parser / Analyzer / Optimizer / Main; Quality Gate = сборка + автотесты.
- **Обязательный Java-scope:** разделы 1–4 (язык и java.lang, консольный ввод-вывод с кириллицей, коллекции java.util, исключения). Бонус: record/enum, дженерики, лямбды, Stream API. Вне рамок: потоки, рефлексия, файлы, сеть, java.time.
- **Оценивание:** поддержка scope, сохранение поведения, полезность диагностики, понятность интерфейса, качество реализации и демо. Веса утверждают организаторы.
- **Сдача:** исходный код с инструкцией, веб-демо (ссылка или воспроизводимая сборка), автотесты с результатом запуска, описание решения (архитектура, scope, ограничения).

2. Сравнение с проектом

Статусы выставлены по чтению кода и истории git. Серверные тесты и сборку клиента здесь запустить не удалось (см. раздел 6).

Требование кейса	Бэкенд	Клиент	Основание
Загрузка .java и разбор в граф	Готово	Мок	POST /api/graph: GraphBuilder, стабильные ID узлов, типы method/var/call/op/condition/loop/try/switch/break/continue/throw/return/raw. Клиент не вызывает fetchGraph: холст рисует INITIAL_NODES. «Импорт» кладёт текст в редактор, граф не меняется.

Требование кейса	Бэкенд	Клиент	Основание
Правка графа, синхронизация в обе стороны	**Готово**	**Нет**	POST /api/graph/mutate: UPDATE_CODE, DELETE_NODE, INSERT_NODE, REORDER, проверка компиляции (14 тестов). В клиенте mutateGraph не вызывается: удаление узла и блоки палитры меняют только локальный граф, правка в Monaco граф не перестраивает.
Сборка Java из графа	**Готово**	**Частично**	Бэкенд возвращает код после мутации. «Экспорт» в клиенте скачивает текст редактора.
Диагностика и Big O	**Готово**	**Мок**	Находит вложенные циклы, дорогие вызовы в цикле, длинные методы (>30 строк), цикломатику (>10), дубли методов. Клиент не вызывает /api/analysis: карточки и метрики из INITIAL_ISSUES.
Оптимизация O(n²) → O(n)	**Частично**	**Сломано**	Шаблонный автофикс есть только для List.contains в цикле. Для вложенных циклов (пример со слайда 5) нужен LLM с ключом или кэшем. Клиент шлёт issueId 'iss-nested-complexity', которого бэкенд не знает (ID вида TYPE:узел) → ошибка → тихий переход на имитацию.
Декомпозиция монолита	**Готово**	**Частично**	Вынос вложенных классов и раскладка static-методов по Parser / Analyzer / Optimizer с проверкой компиляции (4 теста). В клиенте при ошибке оптимизации показываются классы-заглушки.
Quality Gate и сохранение поведения	**Готово**	**Частично**	Режимы EXACT / SORTED_LINES / MASKED, лимит 3000 мс, SSE-логи, verification_report.json (12 тестов). Клиент подключает поток, но кнопка «Запуск» — setTimeout с текстом «12 тестов, 100% PASS»; итог «PASSED» пишется без проверки allPassed; ускорение «5.2x» прошито.
Обязательный Java-scope (разд. 1–4)	**Не доказано**	**—**	Разбор через JavaParser. Лямбды, synchronized, метки, локальные классы дают узел raw. Матрица в ARCHITECTURE_AND_SPEC.md везде пишет «Да», но тестов на каждый раздел нет.
Тестовый монолит ≈1000 строк	**Нет**	**Нет**	samples/Main.java — 172 строки, 4 входных файла. Сценарий на ≈1000 строк в репозитории не представлен.
Кнопка «Дебаг»	**—**	**Нет**	Вызывает alert(); в кейсе не требуется, но обещает функцию, которой нет.

Минимальный комплект сдачи

Элемент	Статус	Комментарий
Исходный код и инструкция запуска	**Готово**	README.md, docker-compose.yml, Maven Wrapper, scripts/run-windows.bat (UTF-8 для кириллицы).
Веб-демо	**Частично**	Воспроизводимая сборка через docker compose up --build описана; в этой проверке не запускалась.
Автотесты и результат запуска	**Частично**	63 теста сервера, CI ci.yml гоняет только сервер. Нет тестов клиента, нет сохранённого отчёта прогона в репозитории.
Описание решения	**Частично**	docs/ARCHITECTURE_AND_SPEC.md и API_CONTRACT.md есть, но спецификация написана как план («для победы»); ограничения не описаны.

3. Ключевые находки

Сильные стороны

- Бэкенд построен по контракту кейса и разложен по пакетам: parser, analyzer, optimizer, sandbox, controller; есть заглушечный профиль **stub** для работы клиента без сервера.
- Quality Gate сделан по существу: отдельные JVM-процессы, таймаут, три режима сравнения вывода, SSE и сохранение отчёта.
- Стабильные ID узлов (путь вида Main.f/s3.t.1) и общая схема слотов для построения и правки графа дают надёжное соответствие «узел ↔ AST».

- Дефекты Big-O привязываются к строке и узлу графа, как требует слайд 5; для contains-в-цикле есть подтверждаемый шаблонный автофикс.
- Клиент визуально близок к концепту слайда 4: граф с автолейаутом, Monaco, вкладки консоли и тестов, панель проблем.

Критичные расхождения

- **Нет сквозного пути Java ↔ граф в интерфейсе.** Это «обязательный финиш» кейса. Подключены только /health, /optimize, /decompose, /verify/stream; не подключены /graph, /graph/mutate, /analysis, /run, /verify (синхронный) и /verify/report.
- **Результаты имитируются.** Если сервер недоступен или оптимизация упала, клиент показывает «12 из 12 тестов», «5.2x (Подтверждено)» и 1240 → 238 мс без пометки о симуляции. Для кейса «диагноз должен быть проверяемым» это самый опасный пункт: жюри может принять мок за результат.
- **Живая оптимизация не срабатывает.** Из-за несовпадения issueId запрос получает ошибку, а клиент молча переходит на имитацию, поэтому проверить честный путь на демо не получится.
- **Quality Gate в клиенте проверяет не то.** В качестве «оригинала» уходит константа INITIAL_JAVA_CODE, а не загруженный пользователем файл; четыре входа зашиты в store, а не берутся из samples/inputs; успех пишется независимо от report.allPassed.
- **Нет монолита на ≈1000 строк.** Заявленный сценарий демо («файл на 1000 строк») ни разу не проверен.

Прочие замечания

- project_state/PROJECT_STATUS.md устарел: описывает ветки client / server / main как разделённые и «100% MVP», хотя код уже слит в main; содержит ссылки на C:\Users\gurzi\….
- На origin остаются ветки asakue-dev, exex37-ver, nepovezlo-develop, client, server; нужно подтвердить, что вся работа слита в main.
- package-lock.json в корне не отслеживается и, видимо, лишний (клиентский lock уже лежит в client/).
- Автофикс вложенных циклов зависит от внешнего LLM (LLM_API_KEY или кэш llm-cache). Без ключа основной пример слайда 5 не оптимизируется автоматически.
- Бэкенд отдаёт узлы try, switch, break, continue, throw, raw, но в client/src/components/canvas/nodes есть только Method, Var, Call, Condition, Loop и Return — остальные типы не имеют собственного вида.

4. План до контрольной точки (08:00 25.10)

Порядок отражает вклад в критерии оценивания: сначала сквозной путь и честность результатов, затем доказательства и оформление.

№	Задача	Результат
1	Подключить /api/graph: при импорте и правке кода (с задержкой) строить холст из ответа; расположение узлов оставить за Dagre.	Загруженный .java превращается в граф; mockData остаётся только для демо без сервера.
2	Подключить /api/analysis вместо INITIAL_ISSUES; брать сложность и ускорение из ответа.	Карточки и Big-O настоящие, клик ведёт к строке и узлу.
3	Подключить /api/graph/mutate для удаления, вставки из палитры и правки узла; обновлять код и граф из ответа.	Граф → Java работает, ошибки компиляции показываются.
4	Исправить issueId в optimize (брать из выбранной карточки); для вложенных циклов добавить шаблонный автофикс или заранее заполнить кэш LLM.	Пример со слайда 5 оптимизируется без внешних зависимостей.
5	Quality Gate: отправлять реальный исходник и входы из samples/inputs; «Запуск» вызывает /api/verify/stream; итог строить по report.allPassed и report.speedup; убрать прошитые 12/12 и 5.2x.	Результат тестов — только от сервера.
6	Режим без сервера пометить «ДЕМО / СИМУЛЯЦИЯ» в интерфейсе или убрать.	Нельзя принять мок за подтверждение.
7	Подготовить 2–3 монолита (один ≈1000 строк), прогнать полный путь, сохранить verification_report.json в репозиторий.	Доказательство для жюри и для сдачи.

№	Задача	Результат
8	Прогнать mvn test на JDK 21 и docker compose up --build на чистой машине; добавить сборку клиента в CI.	Подтверждённая воспроизводимость.
9	Обновить PROJECT_STATUS.md и README: фактический статус, поддержанный scope, ограничения; слить или закрыть лишние ветки.	Пункт «Описание решения» сдачи закрыт честно.
10	До 08:00 25.10 отправить HR ссылки на репозитории; уточнить у организаторов формат и срок сдачи.	Промежуточная оценка не потеряна.

5. Покрытие Java-scope: что проверить

Раздел кейса	Заявлено в docs	Что проверить
1. Язык и java.lang	Да (классы, циклы, switch, try)	Разбор и обратная генерация switch, do-while, break/continue, вложенных классов; узел raw не должен ломать генерацию.
2. Консольный ввод-вывод	Да (Scanner, printf, UTF-8)	Кириллица в stdin/stdout при запуске в песочнице (в последнем коммите добавлен UTF-8 лаунчер для Windows) — нужен тест на русском тексте.
3. Коллекции java.util	Да	Автофикс корректен для List/Set/Map; случаи с мутацией списка в цикле не должны исправляться (тест unsafePatternIsNotAutofixed уже есть).
4. Исключения	Да (throws, свои классы)	Узлы try/catch/finally и throw есть; проверить сохранение checked-исключений при декомпозиции.
Бонус: record, enum, дженерики, лямбды, Stream	Да	Лямбды и вложенные классы уходят в raw; заявление «Да» в матрице нужно либо подтвердить тестами, либо смягчить.

6. Границы проверки

- Анализ статический: прочитаны документация, исходники сервера и клиента, история git. Живой прогон интерфейса не выполнялся.
- Серверные тесты не запускались: на машине установлена только Java 8 (проект требует JDK 21). Количество тестов (63) посчитано по аннотациям @Test, а не по результату прогона.
- Клиент не собирался: в client/ нет node_modules. Бэкенд на localhost:8080 не отвечал.
- Формулировки «не вызывается» и «прошито» относятся к состоянию кода на дату анализа (коммит 6fa4be0).